CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

FULL STACK WEB DEVELOPMENT – PROJECT REPORT

Department: CSE (Artificial Intelligence & Machine Learning)

Subject: Full Stack Web Development

Code: 22CAC11N

Project: PennyTrack – Personal Expense Tracker with Secure Login and AI Financial Assistant

Student Names: A. Bheeshma Shankar & Ch. Pranav

Roll Numbers: 1601-24-748-093 & 1601-24-748-099

Class: B.E. CSM-2, V Semester, III Year (Academic Year 2026-2027)

Faculty Name: Ms. G. Baby Lakshmi Prasanna

Date: 02/11/2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)
CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
(A) GANDIPET, HYDERABAD - 500075

Table of Contents

2. Abstract **2**

3. Problem Statement **3**

3.1 Background 3
3.2 Limitations of Existing Methods 3
3.3 Problem Definition 3
3.4 Scope of the Project 3

4. Objectives **4**

5. System Design **5**

5.1 Requirements Specification 5
5.2 Technology Stack 6
5.3 System Architecture 6
5.4 Application Modules 7
5.5 Database Design 7
5.6 REST API Design 8
5.7 Authentication Flow 9
5.8 Summary Calculations 9

6. Implementation **10**

6.1 Project Structure 10
6.2 Express Server Configuration 10
6.3 Mongoose Schemas 10
6.4 Registration and Login 11
6.5 Authentication Middleware 12
6.6 Transaction CRUD 12
6.7 Monthly Summary using Aggregation 12
6.8 AI Agent — Pennywise 13
6.9 Frontend — API Helper and Routing 13
6.10 Security Measures 14
6.11 How to Run the Project 15

7. Outputs **16**

8. Testing and Validation **22**

9. Results and Findings **23**

10. Conclusion **24**

11. Future Enhancements **25**

References **25**

2. Abstract

Managing personal money is a daily problem for students and young professionals: income arrives in one or two lumps, while spending is scattered across dozens of small payments made through cash, cards and UPI. Without a record, it is impossible to know how much is being saved or which habits are draining the budget.

PennyTrack is a full-stack web application, built on the **MERN stack**, that solves this problem. Each user creates a secure account and records income and expenses with an amount, category, date and note. The application then automatically calculates monthly totals, the remaining balance, the savings rate and a category-wise breakdown of spending using the **MongoDB aggregation framework**. An integrated AI assistant, **Pennywise**, reads the user's own figures and gives practical saving advice; it works offline with built-in rules and can optionally use the **Google Gemini API** for conversational answers.

The frontend is a React 19 single-page application (Vite, React Router, Bootstrap 5). The backend is a RESTful API built with Node.js and Express.js 5, and MongoDB with Mongoose stores the data. Passwords are hashed with **bcrypt (12 salt rounds)**, sessions use signed **JSON Web Tokens** with a 24-hour expiry, and every database query is scoped to the logged-in user so that no user can read or change another user's records. The project was verified with 15 functional test cases covering authentication, CRUD operations, validation, data isolation and the AI module.

Keywords: MERN stack, expense tracker, REST API, JWT authentication, bcrypt, MongoDB aggregation, single-page application, AI financial assistant.

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

3. Problem Statement

3.1 Background

Most people know their monthly income but not their monthly spending. Small daily payments — food delivery, transport, subscriptions, recharges — feel insignificant individually but add up to a large share of the budget. Studies of personal finance consistently show that simply *recording* expenses is the first step towards controlling them, yet few people do it because the available methods are inconvenient.

3.2 Limitations of Existing Methods

Existing method	Limitation	How PennyTrack addresses it
Memory / mental notes	Unreliable; no history; no totals	Every transaction is stored permanently in MongoDB
Paper diary	Hard to search or summarise; easily lost	Month filter, type/category filters and automatic reports
Spreadsheets	Manual formulas; not private when shared; poor on mobile	Totals calculated by the server; per-user login; responsive UI
Banking / UPI apps	Show only that bank's payments; cash is missing; no categories	Records any source of money, including cash, under user-defined categories
Generic finance apps	Complex, ad-supported, or require bank access	Simple, self-hosted, and requires no bank credentials

Table 1: *Comparison of existing methods with the proposed system*

INSTITUTE OF TECHNOLOGY

3.3 Problem Definition

> **Problem:** To design and develop a secure, responsive web application in which every user can record their own income and expenses, view them month by month, filter them by type and category, see automatically calculated totals, balance and savings rate, and receive simple data-driven suggestions to improve their spending habits — while guaranteeing that one user's data is never visible to another.

3.4 Scope of the Project

- **In scope:** user registration and login, transaction CRUD, monthly filtering, category reports, savings-rate calculation, AI-based advice, sample-data loader, responsive UI.
- **Out of scope (future work):** bank or UPI statement import, multi-currency support, shared/family budgets, charts, and production deployment.

4. Objectives

The project was planned to apply every major topic of the Full Stack Web Development course in one working product. The specific objectives were:

#	Objective	Achieved through
O1	Design responsive web pages using HTML5, CSS3 and Bootstrap	Bootstrap 5.3 grid, cards, forms and custom `styles.css`
O2	Implement client-side logic using modern JavaScript (ES6+)	Arrow functions, async/await, destructuring, Fetch API
O3	Build a single-page application with client-side routing	React 19 components, hooks and React Router DOM 7
O4	Develop backend services and RESTful APIs	Node.js + Express.js 5 with 12 REST endpoints
O5	Design MongoDB schemas with validation	Mongoose `User` and `Transaction` models with constraints and an index
O6	Implement secure authentication and authorisation	bcryptjs password hashing, JWT tokens, `requireAuth` middleware
O7	Generate reports using database aggregation	MongoDB `$match`, `$group`, `$cond`, `$sort` pipelines
O8	Integrate an AI assistant that uses the user's own data	Pennywise — rule-based engine with optional Gemini API

Table 2: *Project objectives and how each was achieved*

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979

5. System Design

5.1 Requirements Specification

Functional Requirements

ID	Requirement
FR-01	A new user shall be able to register with name, email and password.
FR-02	A registered user shall be able to log in and log out; the session shall survive a page refresh.
FR-03	A user shall be able to add, view, edit and delete income and expense transactions.
FR-04	A user shall be able to view transactions for any selected month and filter them by type and category.
FR-05	The system shall display total income, total expenses, balance and savings rate for the month.
FR-06	The system shall display category-wise spending sorted from highest to lowest.
FR-07	The AI assistant shall answer questions and give advice based on the user's own data.
FR-08	A user shall never be able to read, modify or delete another user's data.

Table 3: *Functional requirements*

Non-Functional Requirements

- **Security** — passwords never stored in plain text; all data routes protected by JWT; API keys kept on the server only.
- **Usability** — clean, consistent interface; clear error messages; works on phone, tablet and desktop.
- **Performance** — compound index `{ user: 1, date: -1 }` keeps monthly queries fast as data grows.
- **Reliability** — validation on both client and server; AI module falls back to built-in rules if Gemini is unavailable.
- **Maintainability** — separate client and server folders; routes, models and middleware in their own modules.

Hardware and Software Requirements

Category	Requirement
Processor / RAM	Any dual-core processor, 4 GB RAM or more
Operating system	Windows 10/11, Linux or macOS
Runtime	Node.js v20 or later, npm
Database	MongoDB Community Server (local) or MongoDB Atlas
Browser	Any modern browser — Chrome, Edge or Firefox
Development tools	Visual Studio Code, Postman, Git and GitHub, MongoDB Compass

Table 4: *Hardware and software requirements*

5.2 Technology Stack

Layer	Technology	Role in the project
Frontend	React 19, Vite 7, React Router DOM 7, Bootstrap 5.3	User interface, routing between pages, responsive layout
Backend	Node.js 20, Express.js 5, CORS, dotenv, nodemon	REST API, middleware, business logic, configuration
Database	MongoDB, Mongoose 8 ODM	Persistent storage, schema validation, aggregation
Security	bcryptjs, jsonwebtoken	Password hashing and stateless token authentication
AI (optional)	Google Gemini API (gemini-2.0-flash)	Conversational financial advice
Tools	VS Code, Postman, Git & GitHub	Development, API testing, version control

Table 5: *Technology stack*

5.3 System Architecture

PennyTrack follows a **three-tier client–server architecture**. The presentation tier (React) runs entirely in the browser and talks to the application tier only through HTTP requests carrying JSON. The application tier (Express) checks the JWT on every protected request, validates input and applies business rules, and is the only component that communicates with the data tier (MongoDB) and with the external Gemini API. Because the tiers are decoupled, each can be developed, tested and deployed independently.

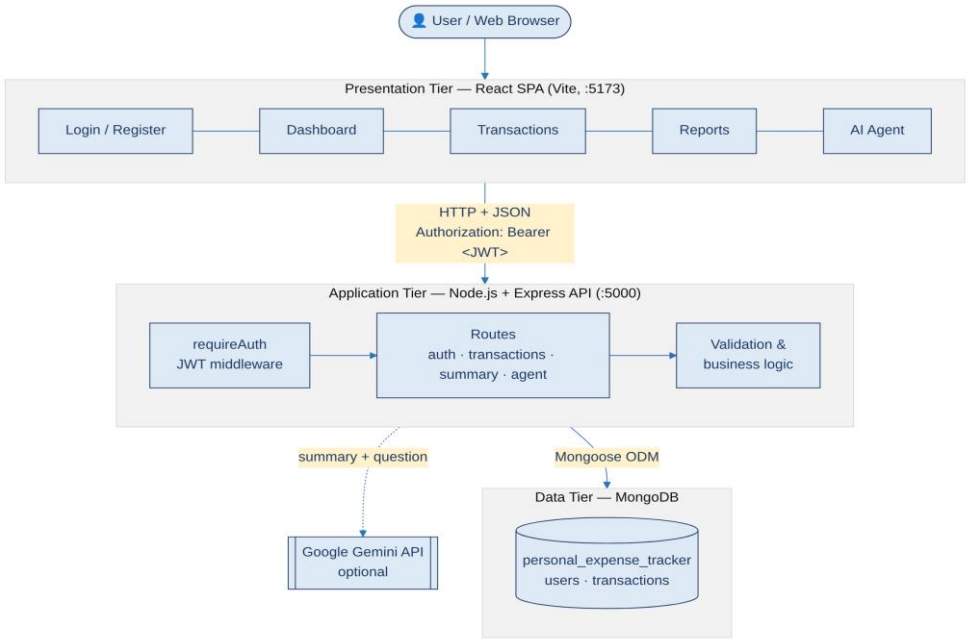


Figure 5.1: *Three-tier architecture of PennyTrack*

5.4 Application Modules

Module	Functionality
Authentication	Register and log in; token stored in the browser; session restored on reload via `/api/auth/me`; log out clears the token.
Dashboard	Four summary cards (income, expenses, balance, savings rate), spending by category, and the five most recent transactions.
Transactions	Form to add or edit a transaction; history table with edit and delete actions; filters by type and category.
Reports	Monthly totals, category-wise spending (highest first), number of transactions and net amount.
AI Agent (Pennywise)	Chat box plus quick tools: monthly review, month-on-month comparison, budget coach and sample-data loader.
Month selector	Global month picker; every page refreshes to show data for the chosen month.

Table 6: *Application modules*

5.5 Database Design

The database `personal_expense_tracker` contains two collections with a **one-to-many** relationship: one user owns many transactions. Each transaction stores the `ObjectId` of its owner, and every query filters on this field — this is what isolates each user's data.

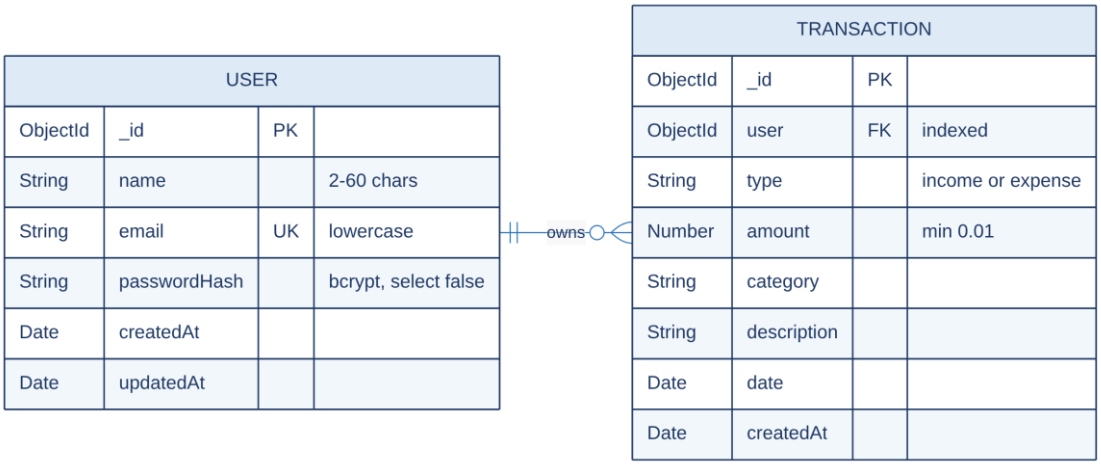


Figure 5.2: *Entity–relationship diagram (User 1 — N Transaction)*

Field	Type	Constraints
name	String	Required, trimmed, 2–60 characters
email	String	Required, unique, lowercase, trimmed, max 254
passwordHash	String	Required; excluded from queries by default (`select: false`)
createdAt / updatedAt	Date	Added automatically (`timestamps: true`)

Table 7: *User collection schema*

Field	Type	Constraints
user	ObjectId → User	Required, indexed
type	String	Enum: `income` | `expense`; required
amount	Number	Required; minimum 0.01
category	String	Required, trimmed
description	String	Optional, trimmed, default empty
date	Date	Required
createdAt / updatedAt	Date	Added automatically

Table 8: *Transaction collection schema*

5.6 REST API Design

All endpoints are prefixed with `/api`. Endpoints marked **Yes** require the header `Authorization: Bearer <token>`.

Method	Endpoint	Auth	Purpose
GET	/api/health	No	Server and database status
POST	/api/auth/register	No	Create account; returns token and user
POST	/api/auth/login	No	Verify credentials; returns token and user
GET	/api/auth/me	Yes	Return the logged-in user
GET	/api/transactions?month=YYYY-MM	Yes	List the user's transactions for a month
POST	/api/transactions	Yes	Add a transaction
PUT	/api/transactions/:id	Yes	Update one of the user's transactions
DELETE	/api/transactions/:id	Yes	Delete one of the user's transactions
POST	/api/transactions/demo-data	Yes	Insert labelled sample data for two months
GET	/api/summary?month=YYYY-MM	Yes	Totals, balance, savings rate, category totals
GET	/api/agent/insights?month=YYYY-MM	Yes	Monthly review text
POST	/api/agent/chat	Yes	Ask Pennywise a question

Table 9: *REST API endpoints*

5.7 Authentication Flow

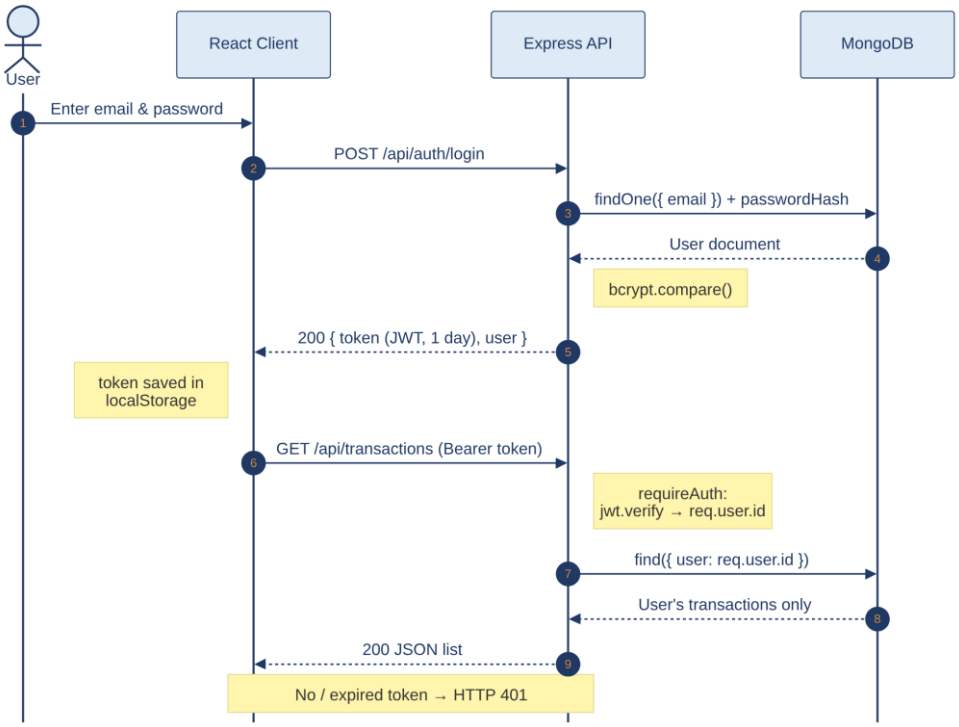


Figure 5.3: *Sequence diagram — login and an authenticated request*

1. The user submits credentials to `/api/auth/register` or `/api/auth/login`.
2. The server validates the input. On registration the password is hashed with bcrypt using 12 salt rounds; on login the stored hash is compared with `bcrypt.compare()`.
3. On success the server returns a JWT signed with `JWT_SECRET` and valid for one day.
4. The client stores the token in `localStorage` and attaches it to every subsequent request.
5. The `requireAuth` middleware verifies the token and places the user id on `req.user`; a missing, invalid or expired token is rejected with **HTTP 401**.
6. Every transaction, summary and AI query is filtered by `req.user.id`.

5.8 Summary Calculations

Metric	Formula
Balance	Total Income − Total Expenses
Savings rate (%)	(Balance ÷ Total Income) × 100; taken as 0 when income is 0
Category total	Sum of expense amounts grouped by category, sorted highest first

Table 10: *Summary formulas*

> **Worked example:** Income ₹30,000 and expenses ₹21,000 in a month give a balance of ₹9,000 and a savings rate of (9,000 ÷ 30,000) × 100 = **30%**.

6. Implementation

6.1 Project Structure

```
personal-expense-tracker-login/
├── client/                      React + Vite frontend
│   ├── index.html
│   └── src/
│       ├── main.jsx             entry point, BrowserRouter
│       ├── App.jsx              pages, routing, API helper
│       └── styles.css           custom theme
└── server/                      Node.js + Express backend
    ├── server.js                app setup, DB connection
    ├── middleware/requireAuth.js
    ├── models/      User.js, Transaction.js
    └── routes/      auth.js, transactions.js, summary.js, agent.js
```

6.2 Express Server Configuration

The server enables CORS only for the React origin, parses JSON bodies, mounts the public authentication router, and protects every other router with the `requireAuth` middleware. For safety it refuses to start if `JWT_SECRET` is missing or shorter than 32 characters.

server/server.js

```
const app = express();
app.use(cors({ origin: process.env.CLIENT_ORIGIN || 'http://localhost:5173' }));
app.use(express.json({ limit: '1mb' }));

app.use('/api/auth', authRouter);                          // public
app.use('/api/transactions', requireAuth, transactionsRouter);
app.use('/api/summary',      requireAuth, summaryRouter);
app.use('/api/agent',        requireAuth, agentRouter);

await mongoose.connect(process.env.MONGODB_URI ||
  'mongodb://127.0.0.1:27017/personal_expense_tracker');
app.listen(PORT, () => console.log(`API running at http://localhost:${PORT}`));
```

6.3 Mongoose Schemas

Validation rules live in the schema, so invalid data is rejected even if a request bypasses the frontend. `select: false` keeps the password hash out of every query unless it is explicitly requested.

server/models/User.js, Transaction.js

```
const userSchema = new mongoose.Schema({
  name:  { type: String, required: true, trim: true, minlength: 2, maxlength: 60 },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  passwordHash: { type: String, required: true, select: false }
}, { timestamps: true });

const transactionSchema = new mongoose.Schema({
  user:     { type: Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  type:     { type: String, enum: ['income', 'expense'], required: true },
  amount:   { type: Number, required: true, min: [0.01, 'Amount must be > 0'] },
  category: { type: String, required: true, trim: true },
  description: { type: String, trim: true, default: '' },
  date:     { type: Date, required: true }
}, { timestamps: true });
transactionSchema.index({ user: 1, date: -1 });
```

6.4 Registration and Login

Registration rejects duplicate emails with **HTTP 409**. Login returns the same generic message for an unknown email and a wrong password, so an attacker cannot discover which emails are registered.

server/routes/auth.js

```
router.post('/register', async (req, res) => {
  const existing = await User.findOne({ email }).select('_id').lean();
  if (existing)
    return res.status(409).json({ message: 'Email already registered.' });
  const passwordHash = await bcrypt.hash(password, 12);
  const user = await User.create({ name, email, passwordHash });
  res.status(201).json({ token: createToken(user), user: publicUser(user) });
});

router.post('/login', async (req, res) => {
  const user = await User.findOne({ email }).select('+passwordHash');
  if (!user || !(await bcrypt.compare(password, user.passwordHash)))
    return res.status(401).json({ message: 'Invalid email or password.' });
  res.json({ token: createToken(user), user: publicUser(user) });
});

const createToken = (user) =>
  jwt.sign({ sub: user._id.toString() }, process.env.JWT_SECRET, { expiresIn: '1d' });
```

6.5 Authentication Middleware

server/middleware/requireAuth.js

```
export default function requireAuth(req, res, next) {
  const authorization = req.headers.authorization || '';
  if (!authorization.startsWith('Bearer '))
    return res.status(401).json({ message: 'Please log in to continue.' });
  try {
    const payload = jwt.verify(authorization.slice(7), process.env.JWT_SECRET);
    req.user = { id: payload.sub };
    next();
  } catch {
    return res.status(401)
      .json({ message: 'Your session is invalid or expired. Please log in again.' });
  }
}
```

6.6 Transaction CRUD

Every query includes `user: req.user.id`. Update and delete use `findOneAndUpdate` / `findOneAndDelete` with **both** the transaction id and the user id, so a user who guesses another record's id still cannot change it.

server/routes/transactions.js

```
// READ – only the logged-in user's records, optionally for one month
router.get('/', async (req, res) => {
  const query = { user: req.user.id };
  if (req.query.month) {
    const r = getMonthRange(String(req.query.month));
    query.date = { $gte: r.start, $lt: r.end };
  }
  res.json(await Transaction.find(query).sort({ date: -1, createdAt: -1 }));
});

// CREATE
await Transaction.create({ ...parsed.value, user: req.user.id });

// UPDATE / DELETE – owner only
await Transaction.findOneAndUpdate({ _id: req.params.id, user: req.user.id },
  parsed.value, { new: true, runValidators: true });
await Transaction.findOneAndDelete({ _id: req.params.id, user: req.user.id });
```

6.7 Monthly Summary using Aggregation

Instead of downloading all records and adding them in JavaScript, the server lets MongoDB compute the totals in a single pipeline. `$cond` splits amounts into income and expense buckets, and a second pipeline groups expenses by category.

server/routes/summary.js

```
const totals = await Transaction.aggregate([
  { $match: { user: userId, date: { $gte: range.start, $lt: range.end } } },
  { $group: { _id: null,
      income:   { $sum: { $cond: [{ $eq: ['$type', 'income']  }, '$amount', 0] } },
      expenses: { $sum: { $cond: [{ $eq: ['$type', 'expense'] }, '$amount', 0] } },
      transactionCount: { $sum: 1 } } }
]);

const categoryTotals = await Transaction.aggregate([
  { $match: { user: userId, type: 'expense',
              date: { $gte: range.start, $lt: range.end } } },
  { $group: { _id: '$category', total: { $sum: '$amount' }, count: { $sum: 1 } } },
  { $sort:  { total: -1 } }
]);
```

6.8 AI Agent — Pennywise

Pennywise works in three steps:

1. **Build context** — the server summarises the logged-in user's current and previous month: income, expenses, balance and category totals.
2. **Rule-based advice** — it identifies the highest-spending category, the change from last month and the savings rate, and drafts a simple 50/30/20-style budget plan.

3. **Optional Gemini call** — if `GEMINI_API_KEY` is set, the summary and the user's question are sent to Gemini for a conversational reply. If the key is missing or the call fails, the rule-based answer is returned, so the feature never breaks.

The Gemini key is read only on the server and is never sent to the browser.

6.9 Frontend — API Helper and Routing

A single `request()` helper attaches the token to every call and converts server errors into readable messages. Protected pages are rendered only when a user is logged in; otherwise every path shows the login page.

client/src/App.jsx

```
async function request(path, options = {}) {
  const token = localStorage.getItem('pennytrack_token');
  const response = await fetch(`${API}${path}`, {
    ...options,
    headers: { 'Content-Type': 'application/json',
               ...(token ? { Authorization: `Bearer ${token}` } : {}),
               ...(options.headers || {}) }
  });
  const data = await response.json().catch(() => ({}));
  if (!response.ok) throw new Error(data.message || 'Request failed.');
  return data;
}

// Routes available after login
<Route path="/"             element={<Dashboard summary={summary} txns={tx} />} />
<Route path="/transactions" element={<TransactionsPage ... />} />
<Route path="/reports"      element={<ReportsPage summary={summary} />} />
<Route path="/agent" element={<AIAgentPage month={month} onDataChanged={loadData} />} />
```

On page load the app calls `/api/auth/me` to restore the session, and `loadData()` fetches the transactions and the summary in parallel with `Promise.all`.

6.10 Security Measures

Threat	Counter-measure
Password theft from database	bcrypt hashing with 12 salt rounds; hash hidden by `select: false`
Unauthorised API access	JWT required on every data route; 24-hour expiry
Accessing another user's records	All queries, updates and deletes filtered by `user: req.user.id`
Email enumeration	Same "Invalid email or password" message for both failure cases
Invalid or malicious input	Validation in the React form, in the route handler and in the Mongoose schema
Weak secret / leaked API key	Server refuses to start with a short `JWT_SECRET`; Gemini key kept in `.env` on the server
Cross-origin requests	CORS limited to the configured client origin

Table 11: *Security measures implemented*

6.11 How to Run the Project

Environment variables: `PORT`, `MONGODB_URI`, `JWT_SECRET` (32+ characters), `CLIENT_ORIGIN`, and optionally `GEMINI_API_KEY` and `GEMINI_MODEL`.

Commands to run the application

```
# Terminal 1 – Backend                        # Terminal 2 – Frontend
cd server                                      cd client
copy .env.example .env   (fill in values)      npm install
npm install                                    npm run dev
npm run dev                                    → open http://localhost:5173
→ API on http://localhost:5000
```

7. Outputs

The screenshots below were captured from the running application and show each module working end to end.

7.1 Register page

A new user enters name, email and password. Client-side checks run before the request is sent; on success the user is logged in and taken to the dashboard.

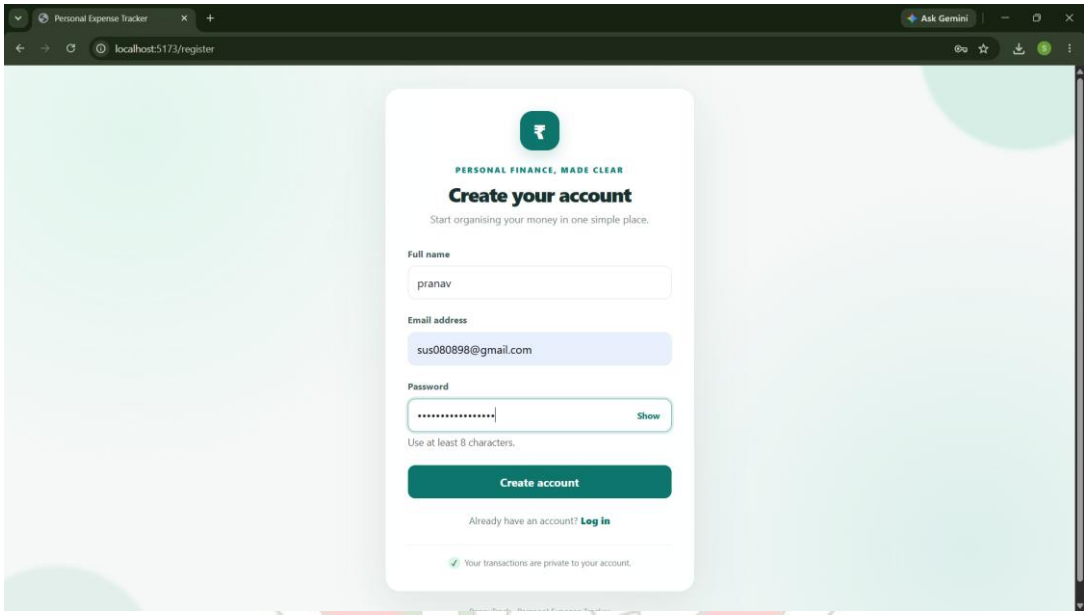


Figure 7.1: *Register page*

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ

7.2 Login page

Existing users sign in with email and password. Incorrect credentials show a single generic error message.

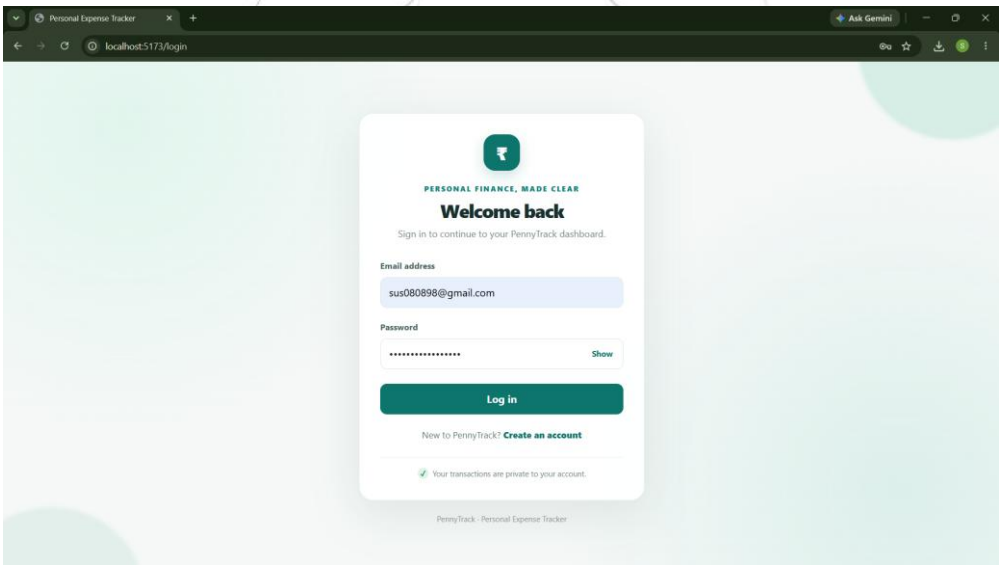


Figure 7.2: *Login page*

7.3 Dashboard

Summary cards show income, expenses, balance and savings rate for the selected month, followed by spending by category and the five most recent transactions.

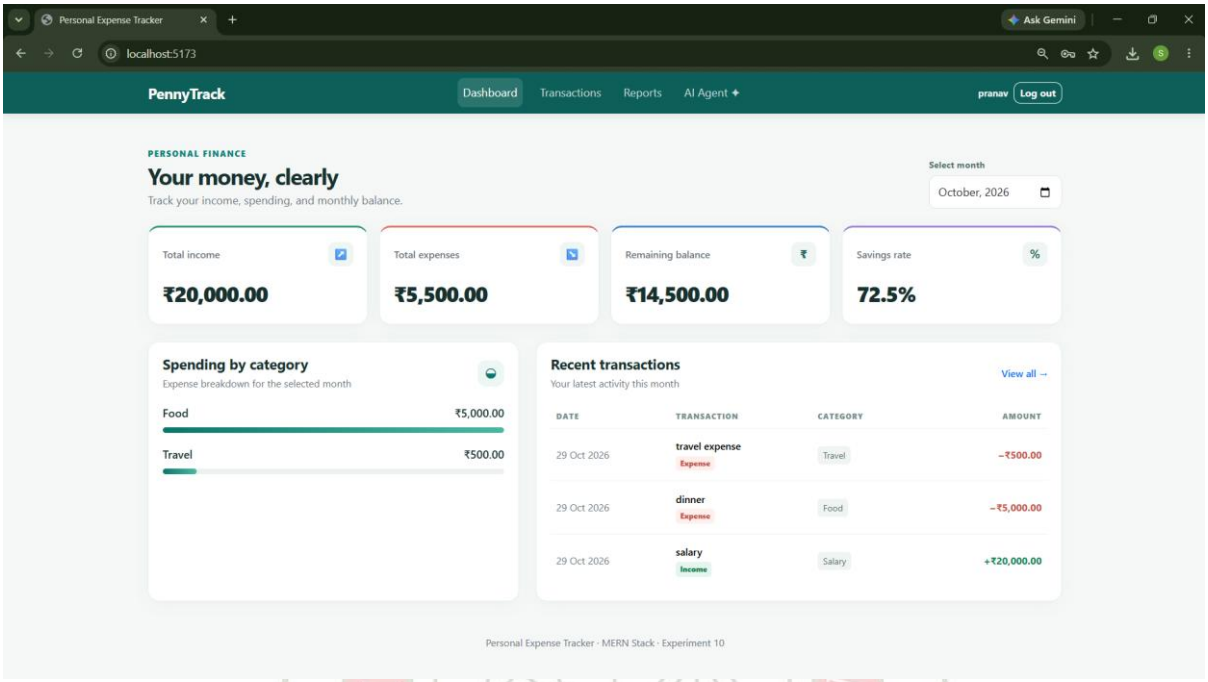


Figure 7.3: *Dashboard*

7.4 Transactions page

The form on the left adds a new transaction; the history table lists all transactions for the month with edit and delete actions.

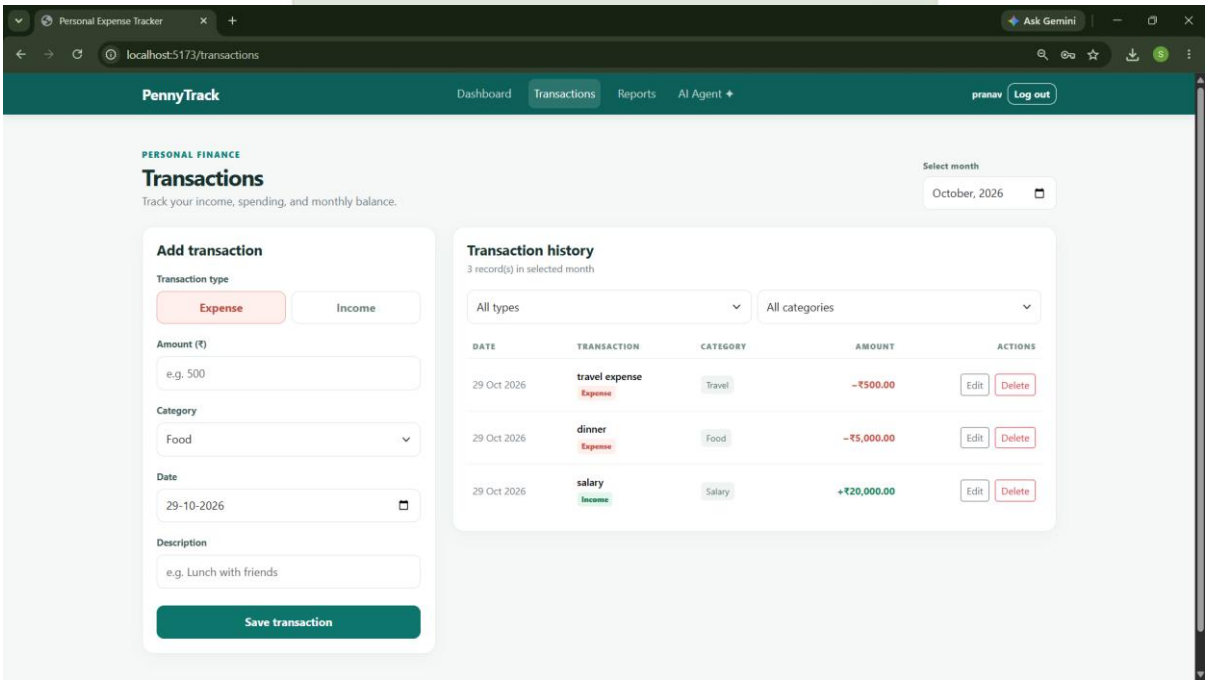


Figure 7.4: *Transactions page*

7.5 Editing and deleting a transaction

Selecting Edit loads the record into the form; Delete asks for confirmation before removing it. The summary is recalculated immediately.

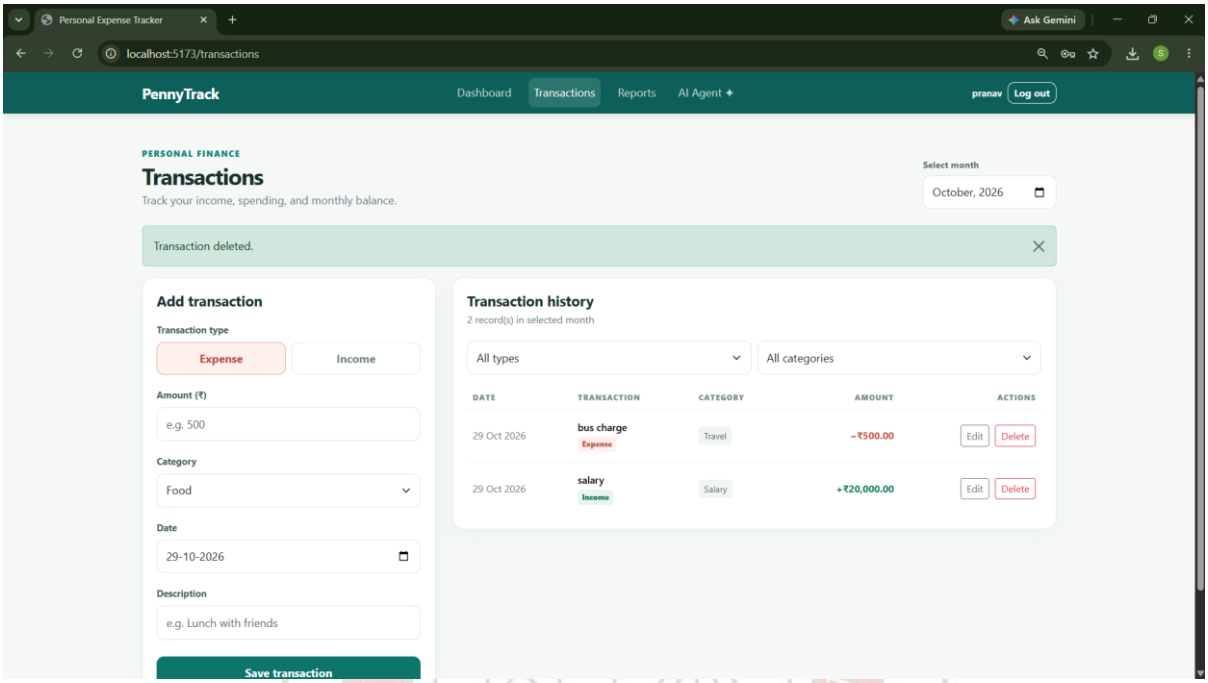


Figure 7.5: *Editing and deleting a transaction*

7.6 Filtering by type and category

Type and category filters narrow the history table to matching records only.

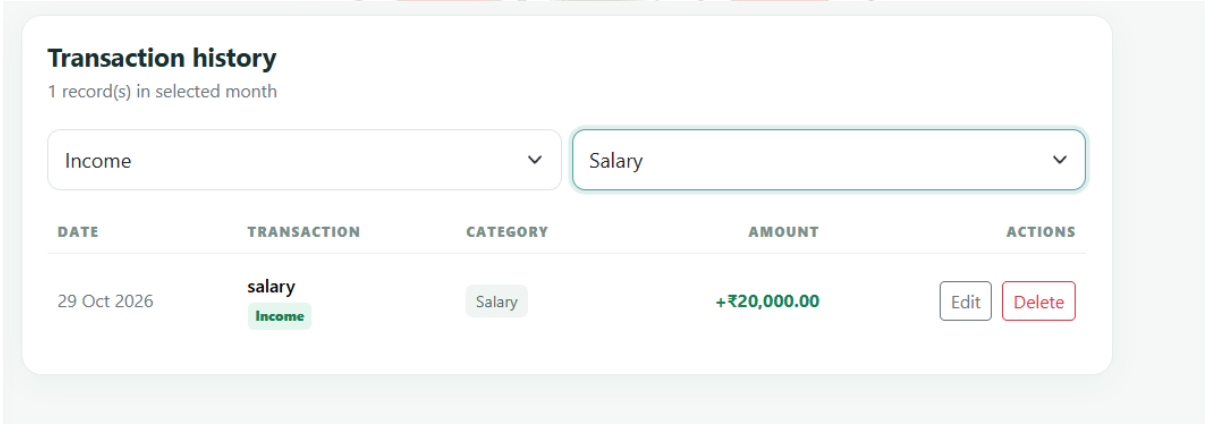


Figure 7.6: *Filtering by type and category*

7.7 Reports page

Category-wise spending sorted from highest to lowest, together with monthly totals and the transaction count.

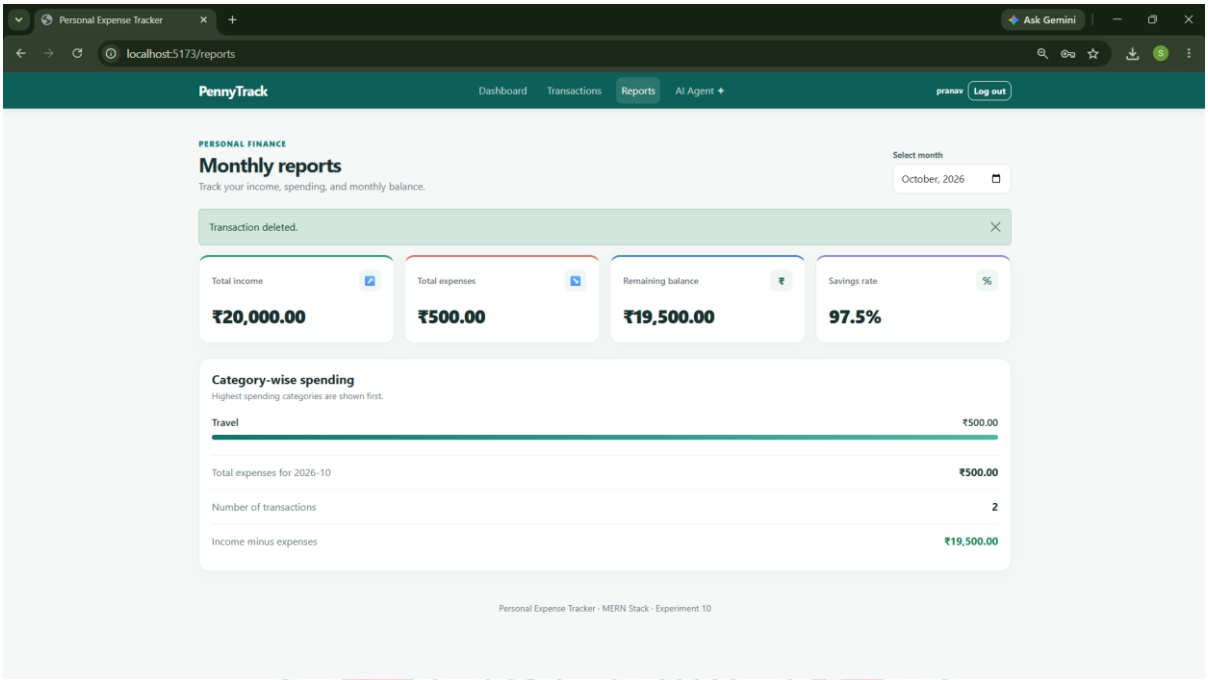


Figure 7.7: *Reports page*

7.8 AI Agent — Pennywise

The chat panel answers questions about the user's own spending; quick tools generate a monthly review, a month-on-month comparison and a budget plan.

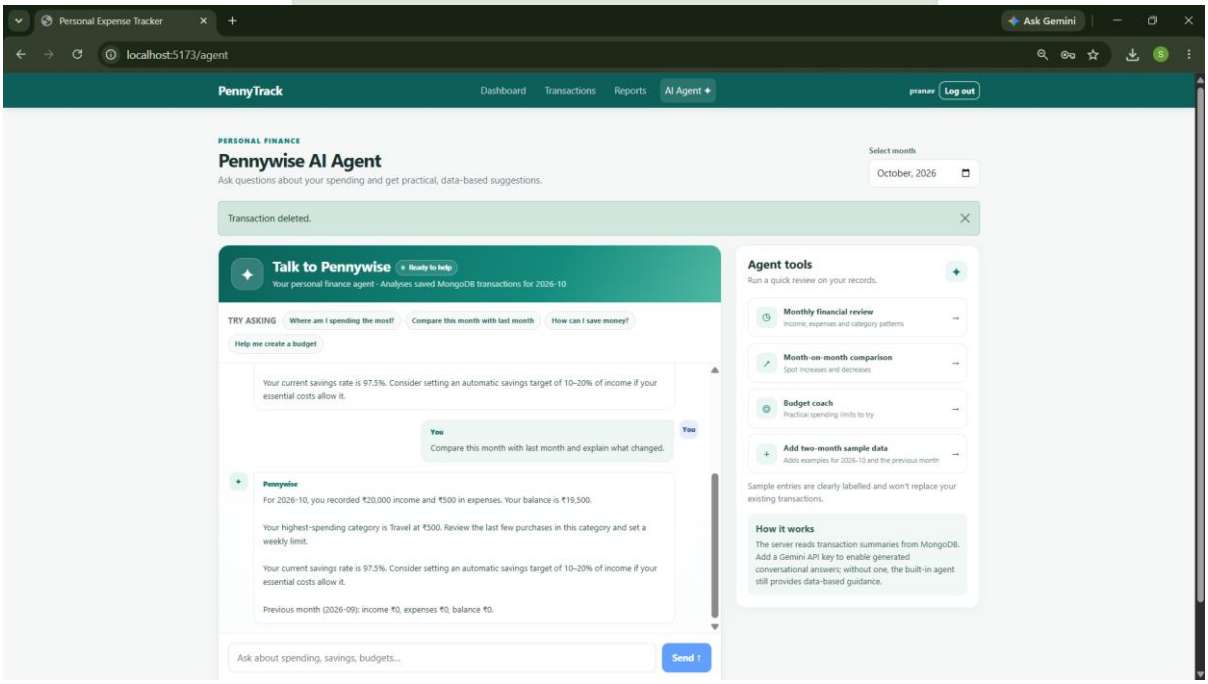


Figure 7.8: *AI Agent — Pennywise*

7.9 MongoDB collections

The users collection stores only a bcrypt hash of the password; each document in the transactions collection holds the owner's user id.

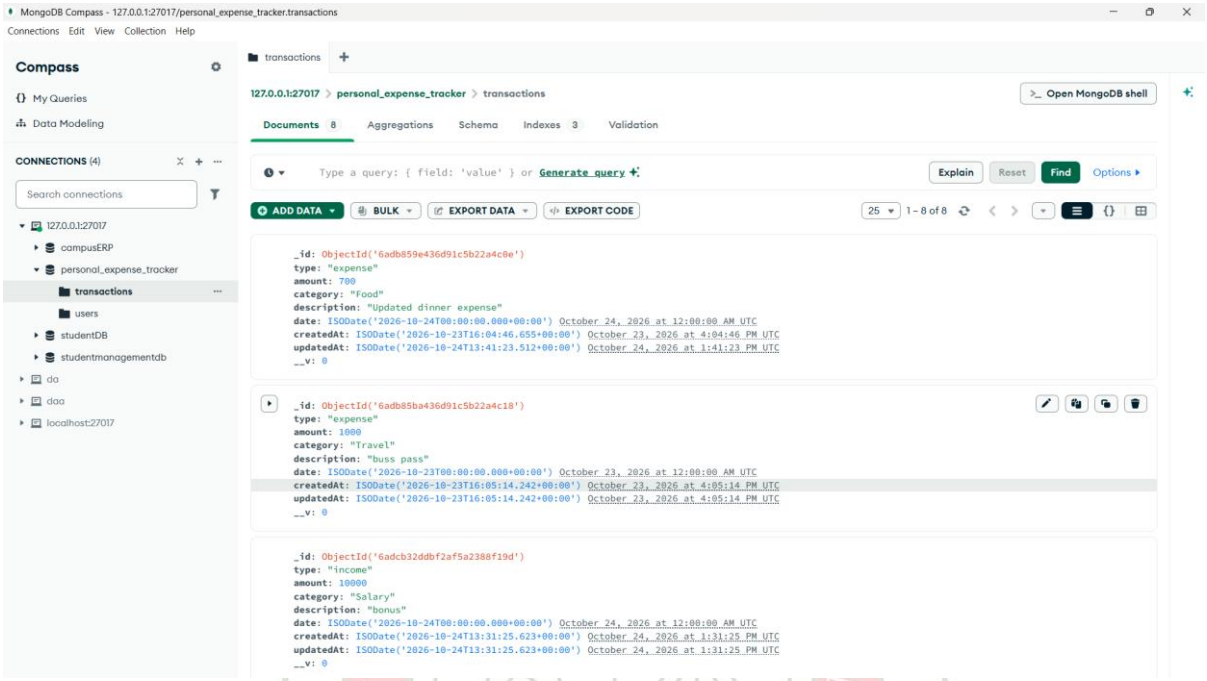


Figure 7.9: *MongoDB collections*

7.10 API testing in Postman

Login returns a token; a protected route succeeds with the token and returns HTTP 401 without it.

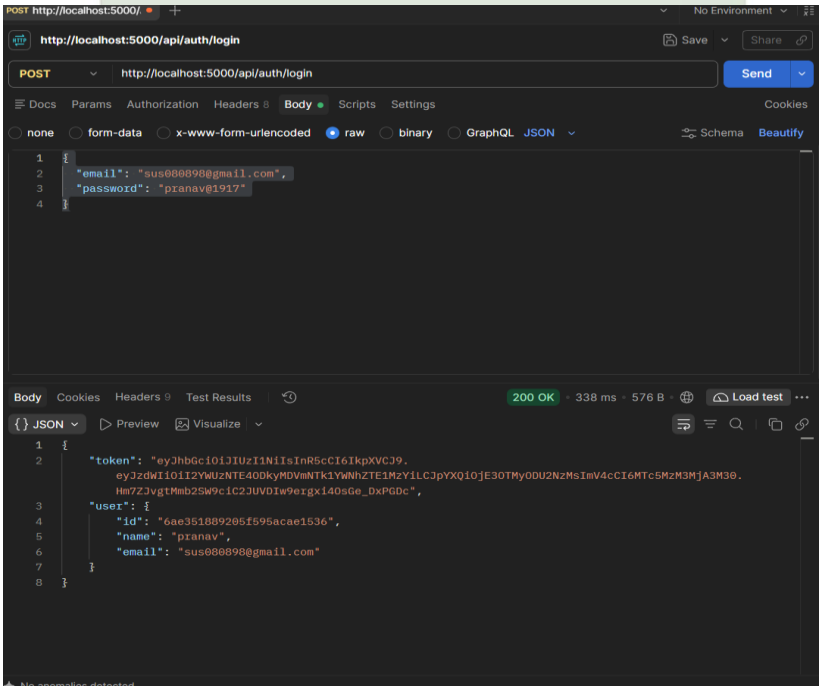


Figure 7.10: *API testing in Postman*

8. Testing and Validation

Testing was carried out at two levels: **API testing** with Postman, to check status codes and responses directly, and **functional testing** through the browser, to check the complete user flow. Each test case below was executed manually against a local MongoDB database.

TC	Scenario	Input / Action	Expected Result	Status
TC-01	Registration	Valid name, email and password	Account created, token stored, dashboard opens	Pass
TC-02	Duplicate email	Register with an existing email	HTTP 409; message asks user to log in	Pass
TC-03	Weak password	Password shorter than 8 characters	Rejected with a validation message	Pass
TC-04	Valid login	Correct email and password	Token returned; dashboard opens	Pass
TC-05	Invalid login	Wrong password	HTTP 401 "Invalid email or password."	Pass
TC-06	Protected route	GET /api/transactions without token	HTTP 401 "Please log in to continue."	Pass
TC-07	Add transaction	Valid income or expense	Saved in MongoDB, listed, totals updated	Pass
TC-08	Amount validation	Amount 0 or empty	Blocked in form / HTTP 400 from server	Pass
TC-09	Edit transaction	Change amount or category	Record updated; summary recalculated	Pass
TC-10	Delete transaction	Confirm delete	Record removed; summary recalculated	Pass
TC-11	Filters	Select type and category	Only matching records displayed	Pass
TC-12	Month selection	Change the month picker	Transactions and summary switch month	Pass
TC-13	Data isolation	Log in as a second user	First user's data is not visible	Pass
TC-14	AI agent	"Where am I spending the most?"	Answer uses the user's category totals	Pass
TC-15	Logout	Click Log out	Token removed; login page shown	Pass

Table 12: *Test cases and results*

Summary: 15 test cases executed, 15 passed. Positive cases confirm that each feature works; negative cases (TC-02, 03, 05, 06, 08) confirm that invalid input and unauthorised access are rejected with the correct status code.

9. Results and Findings

Requirement	Result	Evidence
FR-01, FR-02 Authentication	Achieved	Fig. 7.1, 7.2, 7.10; TC-01 to TC-06, TC-15
FR-03 Transaction CRUD	Achieved	Fig. 7.4, 7.5; TC-07 to TC-10
FR-04 Month and filters	Achieved	Fig. 7.6; TC-11, TC-12
FR-05, FR-06 Reports	Achieved	Fig. 7.3, 7.7
FR-07 AI assistant	Achieved	Fig. 7.8; TC-14
FR-08 Data isolation	Achieved	Fig. 7.9; TC-13

Table 13: *Requirements traceability*

Key Findings

- **Security works as designed.** Passwords are stored only as bcrypt hashes, and every protected route returns HTTP 401 without a valid token.
- **Data is fully isolated.** Because the owner's id is part of every query, a second user cannot see or modify the first user's transactions.
- **Aggregation is efficient.** Totals and category reports are calculated by MongoDB in one round trip rather than in JavaScript on the client.
- **Smooth single-page experience.** Navigation between Dashboard, Transactions, Reports and AI Agent happens without a page reload, and the session survives a refresh.
- **AI is reliable.** Pennywise always responds — with Gemini when a key is configured and with built-in rules otherwise.
- **Responsive layout.** The Bootstrap grid adapts the interface to phones, tablets and desktops.

10. Conclusion

PennyTrack was successfully designed, implemented and tested as a complete MERN-stack application. It meets all eight functional requirements: users can register and log in securely, manage their income and expense records, view monthly summaries and category reports, and receive personalised saving advice from the Pennywise assistant.

The project brought together every layer of full-stack development. On the frontend, React components, hooks and React Router produced a responsive single-page application. On the backend, Express.js provided a structured REST API with middleware-based authentication. MongoDB and Mongoose provided schema validation, indexing and aggregation-based reporting.

Beyond the individual technologies, the project taught important engineering practices: never trusting client input, scoping every query to the authenticated user, keeping secrets on the server, and designing features — such as the AI fallback — so that they degrade gracefully. These principles apply directly to any real-world web application that handles personal data.

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY స్వయం తేజస్విన్ భవ 1979

11. Future Enhancements

The current version is a solid foundation. The following enhancements would make it ready for real-world, production use:

Area	Enhancement	Benefit
Visualisation	Pie, bar and line charts using Chart.js or Recharts	Spending patterns understood at a glance
Budgets	Monthly and per-category limits with alerts	Warns users before they overspend
Automation	Recurring transactions (rent, salary, subscriptions)	Less manual entry
Data import	Bank / UPI statement import with ML auto-categorisation	Complete, effortless records
AI	Spending prediction and anomaly detection	Flags unusual expenses early
Security	Refresh tokens in HTTP-only cookies, rate limiting, email verification, password reset	Stronger protection against attacks
Export	Download reports as PDF or Excel	Easy sharing and record keeping
Real-time	Live updates and notifications with Socket.IO	Instant sync across devices
Mobile	Progressive Web App or React Native app	Native-like use on phones, offline support
Deployment	Vercel/Netlify (client), Render (API), MongoDB Atlas, GitHub Actions CI/CD	Publicly accessible, automatically deployed

Table 14: *Planned future enhancements*

INSTITUTE OF TECHNOLOGY
స్వయం తేజస్వినా భవ

References

[1] React Documentation. Meta Open Source. https://react.dev

[2] Express.js — Fast, unopinionated web framework for Node.js. https://expressjs.com

[3] Node.js Documentation. OpenJS Foundation. https://nodejs.org/docs

[4] MongoDB Manual — Aggregation Pipeline. https://www.mongodb.com/docs/manual/aggregation

[5] Mongoose ODM Documentation. https://mongoosejs.com/docs

[6] Introduction to JSON Web Tokens. https://jwt.io/introduction

[7] bcrypt.js — Optimized bcrypt in JavaScript. https://github.com/dcodeIO/bcrypt.js

[8] Bootstrap 5 Documentation. https://getbootstrap.com/docs/5.3

[9] Vite — Next Generation Frontend Tooling. https://vite.dev

[10] Google AI for Developers — Gemini API. https://ai.google.dev